

Figure 1: I2C and SPI driver before regmap

volatile_reg: This is a user defined function written in driver code, which is called whenever a register is written or read through the cache. Whenever a driver reads or writes a register through the regmap cache, this function is called first, and if it returns 'false' only then is the cache method used; else, the registers are written or read directly, since the register is volatile and caching is not to be used. This is a 'per register' operation callback and is optional.

volatile_table: If a driver does not provide a *volatile_reg* callback, then the *volatile_table* is checked by regmap to see if the register is volatile or not. If the register address lies in the range provided by the *volatile_table* then the cache operation is not used. This is also optional, and the driver can omit its definition and can set it to NULL.

lock: This is a user defined callback written in driver code, which is called before starting any read or write operation. The function should take a lock and return it. This is an optional function—if it is not provided, regmap will provide its own locking mechanism.

unlock: This is user defined callback written in driver code for unlocking the lock, which is created by the lock routine. This is optional and, if it's not provided, will be replaced by the regmap internal locking mechanism.

lock_arg: This is the parameter that is passed to the *lock* and *unlock* callback routine.

fast_io: regmap internally uses mutex to lock and unlock, if a custom lock and unlock mechanism is not provided. If the driver wants regmap to use the spinlock, then *fast_io* should be set to 'true'; else, regmap will use the mutex based lock.

max_register: Whenever any read or write operation is to be performed, regmap checks whether the register address is less than *max_register* first, and only if it is, is the operation performed. *max_register* is ignored if it is set to 0.

read_flag_mask: Normally, in SPI or I2C, a write or read will have the highest bit set in the top byte to differentiate write and read operations. This mask is set in the higher byte of the register value.

write_flag_mask: This mask is also set in the higher byte of the register value.

These are the fields of the struct *regmap_config*, and this configuration is passed to *regmap_init*, which creates the struct *regmap* and returns what can be used in the read and

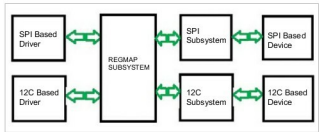


Figure 2: I2C and SPI driver after regmap

write operation.

wr_table, *rd_table* and *volatile_table* are optional tables (used only if the respective callback is not provided) used by regmap for range-checking in case of write and read operations. These are implemented as *struct regmap_access_table*. The following are its fields.

yes_ranges: These are address ranges that are considered as valid ranges.

n_yes_ranges: This is the number of entries in *yes_ranges*.

no_ranges: These are address ranges that are considered as invalid ranges.

n_no_ranges: This is the number of entries in *no_ranges*.

regmap APIs

regmap APIs are declared in *include/linux/regmap.h*. The following are the details of important APIs.

Initialisation routines: The following routine initialises regmap data structures based on the SPI configuration:

```
struct regmap * devm_regmap_init_spi(struct spi_device *spi,
const struct regmap_config);
```

The following routine initialises regmap data structures based on the I2C configuration:

```
struct regmap * devm_regmap_init_i2c(struct i2c_client *i2c,
const struct regmap_config);
```

In the regmap initialisation routine, the *regmap_config* configuration is taken; then the regmap structure is allocated and the configuration is copied to it. The read/write functions of the respective bus are also copied in the regmap structure. For example, in the case of the SPI bus, the regmap read and write function pointer will point to the SPI read and write functions.

After regmap initialisation, the driver can talk to the device using the following routine:

```
int regmap_write(struct regmap *map, unsigned int reg,
unsigned int val);
int regmap_read(struct regmap *map, unsigned int reg,
unsigned int *val);
```

regmap_write: This function is used to write data to the device. It takes in the regmap structure returned during